7.1

将两个线程的语句分别记为：

步骤	T1	步骤	T2
A1	y:=1	B1	x:=1
A2	y:=y+2	B2	x:=x+1
A3	V(S1)	B3	P(S1)
A4	z:=y+1	B4	x:=x+y
A5	P(S2)	B5	V(S2)
A6	y:=z+y	B6	z:=x+z

以每条赋值语句为一个不可分割的执行步骤，P 操作成功通过后才执行其后续语句。

A2、A3 必须先于 B3 通过，B4、B5 必须先于 A5 通过。因此，B4 执行时 y=3，x=2+3=5；A4 执行时 y 仍为 3，故 A4 将 z 赋为 4。x 最终恒为 5。

只需比较 A4、A6、B6 的先后关系，且 A4 必须先于 A6：

关键执行顺序	z 的变化	A6 的计算	最终 (x,y,z)
B6→A4→A6	2→7→4	y=4+3=7	(5,7,4)
A4→A6→B6	2→4→9	y=4+3=7	(5,7,9)
A4→B6→A6	2→4→9	y=9+3=12	(5,12,9)

相应的完整执行顺序各取一种：

(5,7,4)：

```
A1 A2 A3 B1 B2 B3 B4 B5 B6 A4 A5 A6
```

(5,7,9)：

```
A1 A2 A3 A4 B1 B2 B3 B4 B5 A5 A6 B6
```

(5,12,9)：

```
A1 A2 A3 A4 B1 B2 B3 B4 B5 A5 B6 A6
```

这三类覆盖逐语句交错执行的全部结果。线程优先级相同不能确定具体交错顺序。

若赋值语句内部的读、计算、写也允许交错，B6 还可能先读取旧 z=2，暂存结果 7，待 A4 写入 z=4 后再把 7 写回，另外产生两种结果：

公共前缀：A1 A2 A3 B1 B2 B3 B4 B5

B6 读 z=2 并算得 7 → A4 写 z=4 → A5 → A6 写 y=7 → B6 写 z=7
最终：(5,7,7)

B6 读 z=2 并算得 7 → A4 写 z=4 → A5 → B6 写 z=7 → A6 写 y=10

最终：(5,10,7)

此时共有五种结果：(5,7,4)、(5,7,7)、(5,7,9)、(5,10,7)、(5,12,9)。

7.2

方法一能够正确同步；方法二不能保证正常运行，可能发生死锁。初始化取 n=5，mutex=1，fullBuffers=0，emptyBuffers=5，offset=0。

方法一先取得可用资源，再进入缓冲区临界区：

- 生产者先 P(emptyBuffers)，保证存在空位置，再 P(mutex) 写数据并更新 offset。
- 消费者先 P(fullBuffers)，保证存在有效数据，再 P(mutex) 读数据并更新 offset。
- mutex 保证缓冲区操作及 offset 更新互斥；生产者和消费者不会因为等待空位或数据而占住 mutex。
- 完成缓冲区操作后释放 mutex，并用 V(fullBuffers) 或 V(emptyBuffers) 通知对方。在正常的线程调度和操作最终完成的条件下，可以持续推进，且 0≤offset≤5。

方法二先持有 mutex，再执行可能阻塞的资源申请，会出现以下两类死锁。

缓冲区状态	已发生的操作	对方随后执行	结果
空，fullBuffers=0	消费者 P(mutex) 成功，随后 P(fullBuffers) 阻塞	生产者 P(mutex) 阻塞，无法写入并 V(fullBuffers)	消费者等待数据，生产者等待消费者持有的锁
满，emptyBuffers=0	生产者 P(mutex) 成功，随后 P(emptyBuffers) 阻塞	消费者 P(mutex) 阻塞，无法读取并 V(emptyBuffers)	生产者等待空位，消费者等待生产者持有的锁

P(fullBuffers) 或 P(emptyBuffers) 阻塞并不会自动释放另一个信号量 mutex。方法二在缓冲区未满也非空时可能暂时运行，但不能保证以后不进入上述死锁状态。正确次序应为“P 资源计数信号量 →P 互斥信号量 → 操作缓冲区 →V 互斥信号量 →V 对方资源计数信号量”。

offset 的更新及对应位置的读写必须包含在同一临界区内；如果 offset 作为缓冲区当前占用量／栈顶位置，可在取出元素时先定位到 offset−1，再执行 offset–。若要求 FIFO 环形队列，应分别维护写指针 in、读指针 out，均对 5 取模，不能只用一个增减的 offset 同时表示两种位置。

7.3

设 waiting 为尚未被柜员接待的顾客计数信号量，mutex 保护取号计数器及等待队列 Q。每名顾客有独立的 called、arrived、finished 信号量，避免叫号时唤醒错误的顾客。启动 n 个柜员线程执行 Teller_Service(id)。

```
semaphore mutex = 1;
semaphore waiting = 0;
int nextTicket = 1;
FIFOQueue<Customer*> Q;

struct Customer {
```

```
    int ticket;
    int counter;
    semaphore called = 0;
    semaphore arrived = 0;
    semaphore finished = 0;
};

Customer_Service() {
    Customer c;
    P(mutex);
    c.ticket = nextTicket;
    nextTicket++;
    Q.push(&c);
    V(mutex);
    V(waiting);

    P(c.called);
    到达 c.counter 号柜台;
    V(c.arrived);
    P(c.finished);
    离开银行;
}

Teller_Service(int id) {
    while (true) {
        P(waiting);
        P(mutex);
        Customer *c = Q.front();
        Q.pop();
        c->counter = id;
        叫号 (c->ticket, id);
        V(c->called);
        V(mutex);

        P(c->arrived);
        为该顾客办理业务;
        V(c->finished);
    }
}
```

取号与入队在同一临界区中完成，FIFO 队列保证按号码先后取出；出队与叫号也在同一临界区中

完成，保证叫号顺序。柜员等待顾客到达及办理业务时不持有 mutex，因此最多可有 n 位柜员并发服务。顾客等到自己的 finished 后才结束，排队期间其请求对象始终有效。

7.4

取 N=250000，每个整数占 4 B，每个文件大小为 1000000 B，约 1 MB。数据采用 32 位无符号整数的小端二进制表示，取值范围为 1–1000000。

使用两个 pthread 屏障，参与者均为 8 个 Map 线程和 1 个 Reduce 线程：

- start_barrier 的计数为 9，所有工作线程就绪后，Map 线程开始并发读取各自文件。
- Map i 将局部最大值写入独占的 a[i] 后到达 done_barrier；Reduce 也在该屏障等待，只有全部 8 个 Map 都完成后才计算全局最大值。
- 各 Map 写不同数组元素，Reduce 仅在完成屏障之后读取，因而不需要额外互斥锁；屏障提供同步及所需的内存可见性。
- Map 遇到文件错误时仍到达完成屏障，Reduce 统一检查错误，避免其他线程永久等待。

数据预先生成，文件 generate_data.py：

```
import random
import struct
from pathlib import Path

N = 250000
rng = random.Random(20261028)
root = Path('data')
root.mkdir(exist_ok=True)
for i in range(8):
    values = [rng.randint(1, 1000000) for _ in range(N)]
    (root / f'map{i}.bin').write_bytes(struct.pack(f'<{N}I', *values))
```

程序文件 mapreduce.cpp：

```
#include <pthread.h>
#include <cstdio>
#include <cstdlib>
#include <cstdint>
#include <climits>
#include <string>
using namespace std;

const int M = 8;
const size_t N = 250000;
static_assert(CHAR_BIT == 8 && sizeof(uint32_t) == 4);

struct node {
    int id;
```

```
    const char *path;
} task[M];
struct result {
    uint32_t value;
    size_t count;
    bool ok;
} a[M];
pthread_barrier_t start_barrier, done_barrier;
int status = 0;

void wait_barrier(pthread_barrier_t *b) {
    int r = pthread_barrier_wait(b);
    if (r != 0 && r != PTHREAD_BARRIER_SERIAL_THREAD) exit(1);
}

void *Map(void *arg) {
    node *p = static_cast<node *>(arg);
    uint32_t maxn = 0;
    size_t count = 0;
    bool ok = true;
    wait_barrier(&start_barrier);
    FILE *f = fopen(p->path, "rb");
    if (!f) ok = false;
    else {
        unsigned char buf[16384];
        size_t len;
        while ((len = fread(buf, 1, sizeof(buf), f)) != 0) {
            if (len % 4 != 0) { ok = false; break; }
            for (size_t i = 0; i < len; i += 4) {
                uint32_t x = uint32_t(buf[i])
                    | (uint32_t(buf[i + 1]) << 8)
                    | (uint32_t(buf[i + 2]) << 16)
                    | (uint32_t(buf[i + 3]) << 24);
                if (x < 1 || x > 1000000) ok = false;
                if (x > maxn) maxn = x;
                ++count;
            }
        }
        if (ferror(f)) ok = false;
        fclose(f);
    }
```

```
    if (count != N) ok = false;
    a[p->id] = {maxn, count, ok};
    wait_barrier(&done_barrier);
    return nullptr;
}

void *Reduce(void *) {
    wait_barrier(&start_barrier);
    wait_barrier(&done_barrier);
    uint32_t maxn = 0;
    for (int i = 0; i < M; ++i) {
        if (!a[i].ok) {
            fprintf(stderr, "Invalid input: %s\n", task[i].path);
            status = 1;
        }
        if (a[i].value > maxn) maxn = a[i].value;
    }
    if (status == 0) {
        for (int i = 0; i < M; ++i)
            printf("Map %d: count=%zu, max=%u\n",
                   i, a[i].count, unsigned(a[i].value));
        printf("Global max: %u\n", unsigned(maxn));
    }
    return nullptr;
}

int main(int argc, char *argv[]) {
    if (argc != 1 && argc != M + 1) return 1;
    string path[M];
    for (int i = 0; i < M; ++i) {
        path[i] = argc == 1 ? "data/map" + to_string(i) + ".bin"
                            : argv[i + 1];
        task[i] = {i, path[i].c_str()};
    }
    if (pthread_barrier_init(&start_barrier, nullptr, M + 1) != 0)
        return 1;
    if (pthread_barrier_init(&done_barrier, nullptr, M + 1) != 0) {
        pthread_barrier_destroy(&start_barrier);
        return 1;
    }
    pthread_t map[M], reduce;
```

```
    if (pthread_create(&reduce, nullptr, Reduce, nullptr) != 0)
        exit(1);
    for (int i = 0; i < M; ++i)
        if (pthread_create(&map[i], nullptr, Map, &task[i]) != 0)
            exit(1);
    for (int i = 0; i < M; ++i) pthread_join(map[i], nullptr);
    pthread_join(reduce, nullptr);
    pthread_barrier_destroy(&done_barrier);
    pthread_barrier_destroy(&start_barrier);
    return status;
}
```

在 Linux 环境下编译与运行：

```
python3 generate_data.py
g++ -std=c++17 -O2 -Wall -Wextra -pthread mapreduce.cpp -o mapreduce
./mapreduce
```

使用上述随机种子生成数据后，运行结果为：

```
Map 0: count=250000, max=999997
Map 1: count=250000, max=999993
Map 2: count=250000, max=999997
Map 3: count=250000, max=999998
Map 4: count=250000, max=999991
Map 5: count=250000, max=999997
Map 6: count=250000, max=999995
Map 7: count=250000, max=999998
Global max: 999998
```

每个 Map 对 N 个整数各遍历一次，Reduce 比较 8 个局部最大值；总工作量为 O(8N)，归约阶段为 O(8)。主线程在所有工作线程结束后销毁两个屏障。